Week 11. Natural Language Processing (NLP) Basics

Hwanjun Song
KAIST, Dept. of ISE

Today's Contents

- **Key Challenge in NLP <<**
- Text Vectorization
- Word2Vector: CBOW and Skip-gram
- Bidirectional Autoencoding (BERT)
- NLP with Pytorch

Natural Language (Text)

- Human language, we call text, is a representative **unstructured data**
  - Information doesn't have a pre-defined format and organization (e.g., matrix, vector), making it more challenging to handle for ML and DL

Key	URL	LC	L#	C#	TS	DM	Links
A1	A	2	1	2	Xx	Yy	A4,..
A2	B	0	1	2	Xx	Yy	A8,..
A3	C	2	1	3	Xx	Yy	A1,..
A4	D	1	2	3	Xx	Yy	A5,..
A5	E	3	2	4	Xx	Yy	A7,..
A6	F	1	2	4	Xx	Yy	A5,...

"n-dimensional vector"



"3-dimensional Tensor"

Text: "I like playing football with my friend and dislike (…)"

Q: How to process this unstructured data for further processing (e.g. input to ML, DL models)?

Key Properties of Text

- **Low Readability**: Machine cannot understand raw text data directly…
  - Machine only can read numerical format of data
- **High Dimension**: Large vocabularies in text lead to datasets with very high dimension, like more than 100K dimension in LLMs
- **Severe Noise**: Natural text inherently contains feature noise from grammatical errors, broken text, typos, etc

"I like played footballs with my friend and dislike (…)"

"I like play football with my frnd and dislike (…)"

Most importantly, **machine-readable** transformation of raw text is required
We call this "**Vectorization**"

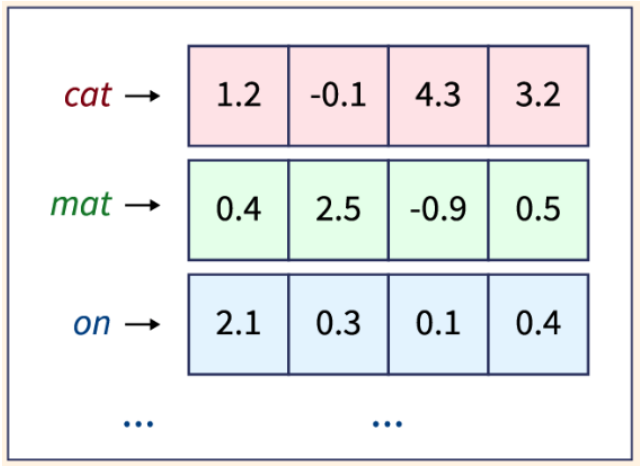
KAIST

Today's Contents

- Key Challenge in NLP
- **Text Vectorization <<**
- Word2Vector: CBOW and Skip-gram
- Bidirectional Autoencoding (BERT)
- NLP with Pytorch

Representing Text in Vectors

- Text vectorization is the process of converting textual data into **numerical representation**, typically to **vectors** (sometimes called **embedding**)
- What advantages?
  - Makes the text into machine readable numerical input
  - Reducing dimensionality with advanced vectorization techniques (w. DL model)
  - Normalizes text data by removing noise



Text Vectorization

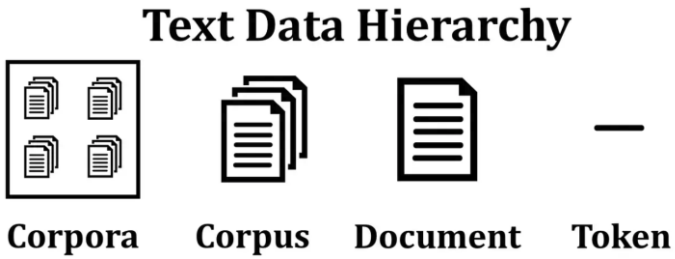
KAIST

Key Terms in NLP

- **Tokens** are the smallest linguistic expression
  - Tokens can be obtained via ”tokenization”

”I am learning deep learning”
- Word tokenization → “I”, “am”, “learning”, “deep”, “learning
- Sub-word tokenization → “I”, “am”, “learn”, “ing”, “deep”, “learn”, “ing”

- **Document** is a sequence of N tokens denoted by d=(t_1, t_2, … t_N), where t_N is the N-th token in the sequence
- **Corpus** is a collection of M documents denoted by D = (d_1, d_2, …, d_M)
- Similarly, **Corpora** is a collection of multiple Corpus



- **Vocabulary**: the set of all tokens in our Corpora

Objective of Text Vectorization

- Given a vocabulary, a set {t_1, t_2, …, t_V}, our goal is to associate each token with a representation, which can explain the meaning token
- Three objectives:
  - **Identifies** which **words** are presented in the input text
  - Capture the **similarity** between words, including syntax and semantics
  - Helps us **solve NLP problems** like word prediction, text summarization, sentiment analysis, etc
- Remark: we can interchangeably use “word representation,” “word vector,” and ” word embedding”; similarly for “word” and “token”

KAIST

One-hot Embedding

- Traditional way to represent tokens as categorical symbols with a unique integer (either 1 or 0) for each word
- Suppose that we have vocabularies with matched symbols:
  - {movie = 1, hotel = 2, apple = 3, movies = 4, art = 5}
- Equivalent to represent tokens as 1-hot vectors:

$$
\begin{aligned}
\text{movie} &= [1, 0, 0, 0, 0] \\
\text{hotel} &= [0, 1, 0, 0, 0] \\
&\dots \\
\text{art} &= [0, 0, 0, 0, 1]
\end{aligned}
$$

KAIST

Limitation of One-hot Embedding

- Not very informative and too high dimensional feature
  - No frequency information that could be different in a sentence
  - Weird to consider all pairs of tokens to have the same distance

$$|| \text{movie} - \text{hotel} || = || \text{movie} - \text{art} ||$$

$$|| [1, 0, 0, 0, 0] - [0, 1, 0, 0, 0] || = || [1, 0, 0, 0, 0] - [0, 0, 0, 0, 1] ||$$

Where || || is the Euclidean distance

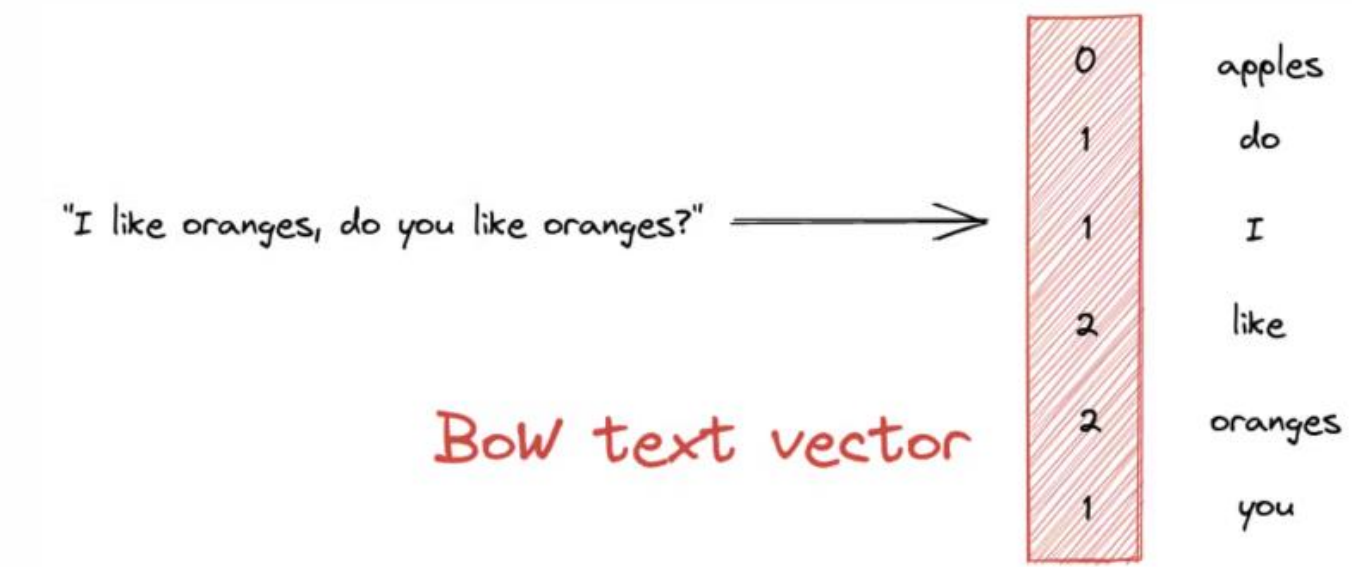
  - Cannot distinguish the semantic different of "art movie" and "movie art" to be the same (the order of word appearance)

Representation = [1, 0, 0, 0, 1]

KAIST

BoW: Bag of Words

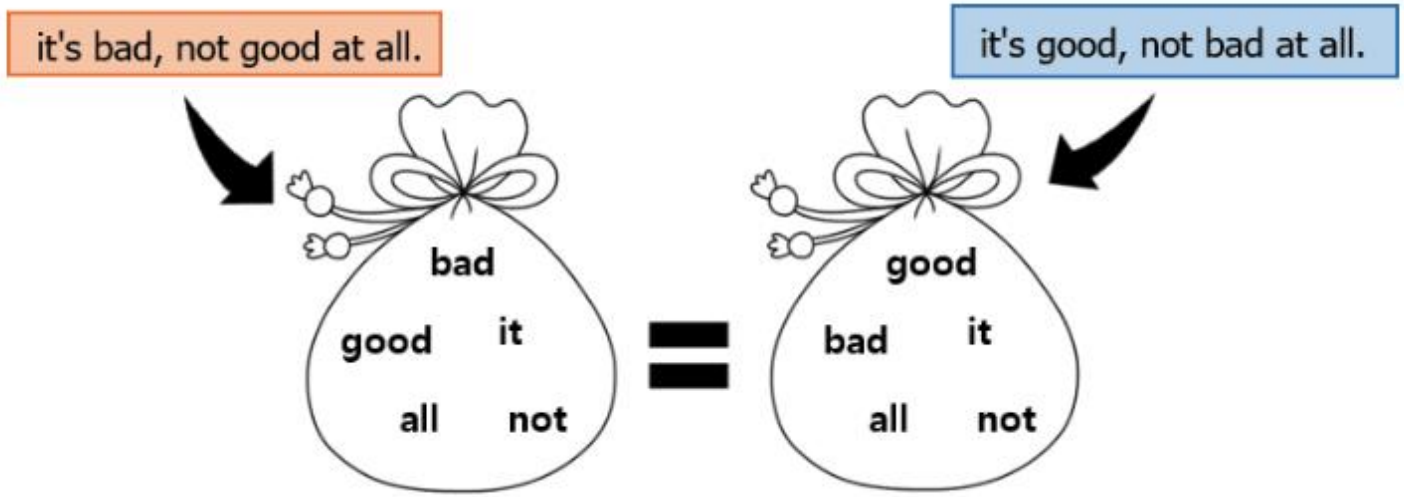
- Another simplest vectorization methods for text, representing the frequency of unique tokens appearing in the given text



- No assumption of the orthonormality of the vector like one-hot but…
  - Still sparse representation; and cannot capture semantical differences

Limitations

- The primary drawback of one-hot embedding and bag-of-words approaches is their inability to capture the difference of word ordering



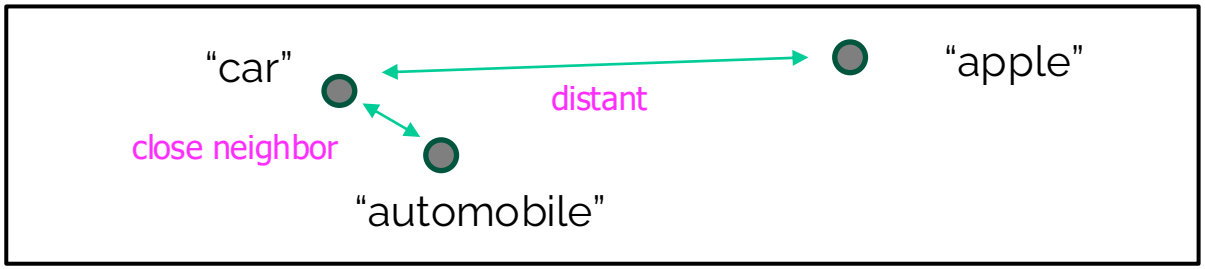
Resulting in exactly the same word embeddings!

- High Sparsity: more than tens of thousands of dimensions per vector
- No Semantical Difference; the meaning of || movie – hotel || ?

Better Solution: Sparse to Dense Embedding

- **Dense vector** of a word is a embedding of that word, usually of much lower dimensions than the vocabulary size; is trained by ML models
- **Key Advantage** from ML training:
  - Has lower dimensional vectors, which is better input to ML/DL models
  - Can capture the semantical difference between words
- **Goal**: Making something like this …
  - ”Car” and “automobile” are synonyms, but are distinct dimensions in BoW

    Car = [1, 0, 0, 0, 0], Automobile = [0, 1, 0, 0, 0], apple = [0, 0, 1, 0, 0]
  - With dense vector:



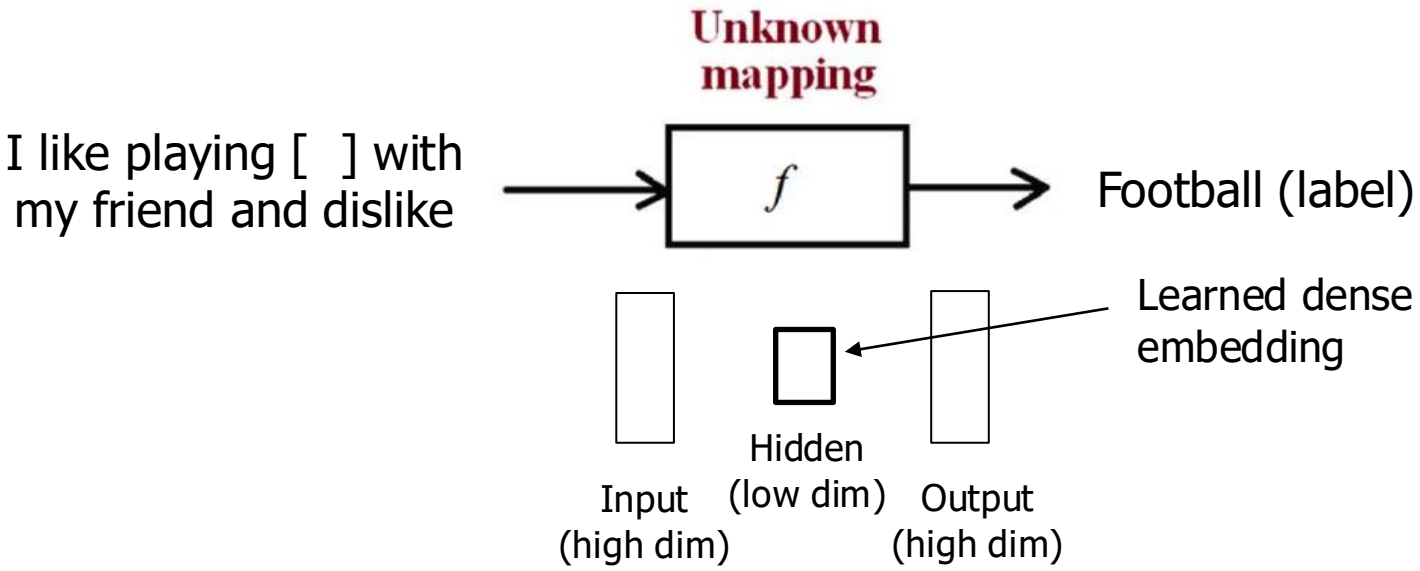
Multi-dim embedding space

KAIST

How to Make Dense Embeddings?

- Let's transform the one-hot embedding of each word to dense embedding via ML/DL approach – **linear** or **non-linear transform** to lower dim
  - E.g., Car = [1, 0, 0, 0, 0] to [0.2, 0.4]
- How?
  - Solving a supervised learning of token prediction using a NN model

A data point: I like playing football with my friend and dislike



Many Variations and Developments

- W2V: Word2Vec Approaches <- using Shallow NNs
  - e.g., CBOW (Continuous Bag of Words), Skip-Gram, etc

- Transformer-based Approaches <- using Deep Transformers
  - e.g., BERT: Bi-directional Masked Autoencoding

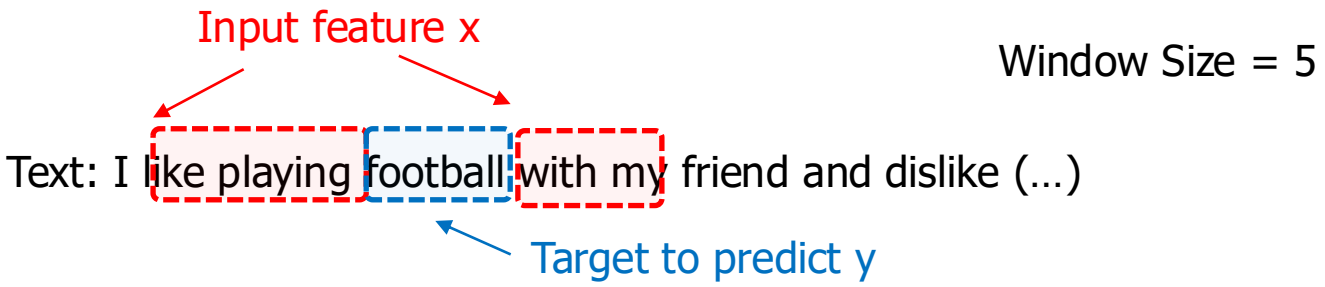
Today's Contents

- Key Challenge in NLP
- Text Vectorization Basic
- **Word2Vector: CBOW and Skip-gram <<**
- Bidirectional Autoencoding (BERT)
- NLP with Pytorch

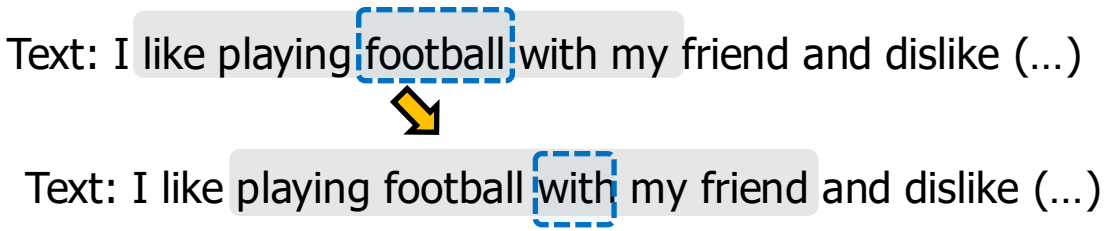
Word2Vec: CBOW (Continuous Bag of Words)

- **Key Idea**: Trains a shallow NN on a binary token prediction task
- Step 1. Preparing a training datasets with <input text, target label>

Use running text as implicitly supervised training data



- Let the neighborhood be the sliding window of size K, moving to right from left in text, generating more x, y pairs



KAIST

Word2Vec: CBOW (Continuous Bag of Words)

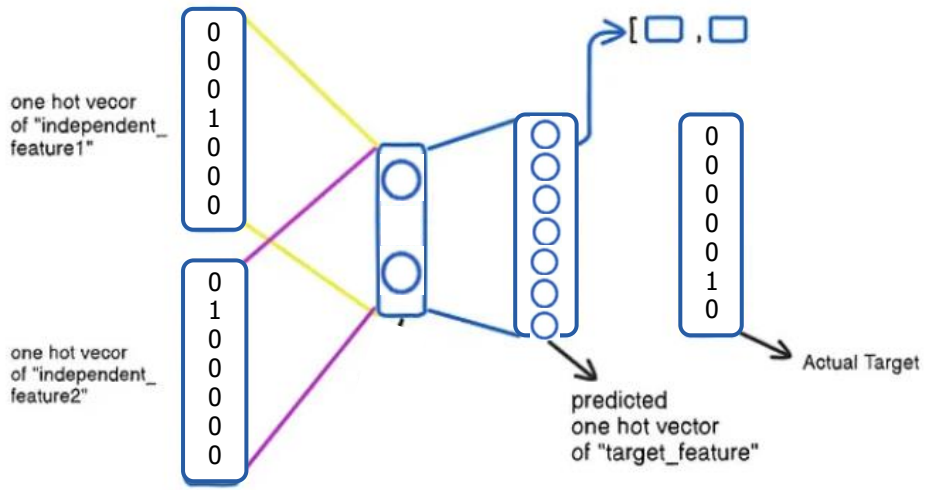
- Step 1. Preparing a training datasets with <input text, target label>
  - Converts all the tokens into one-hot vectors via one-hot encoding

Neighborhood:
[like, playing, with, my] =

Target: [football]

like	0, 0, 0, 1, 0, 0, 0
playing	0, 1, 0, 0, 0, 0, 0
with	1, 0, 0, 0, 0, 0, 0
my	0, 0, 0, 0, 0, 0, 1
football	0, 0, 0, 0, 0, 1, 0

- Step 2. Preparing a NN model: 1 input, 1 hidden, 1 output layer

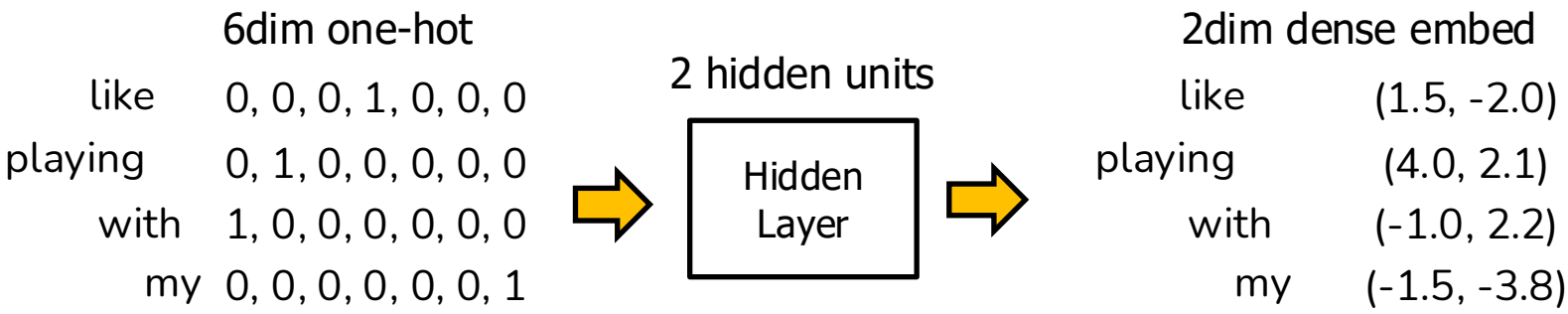


of units in input layer
>> # of units in hidden layer

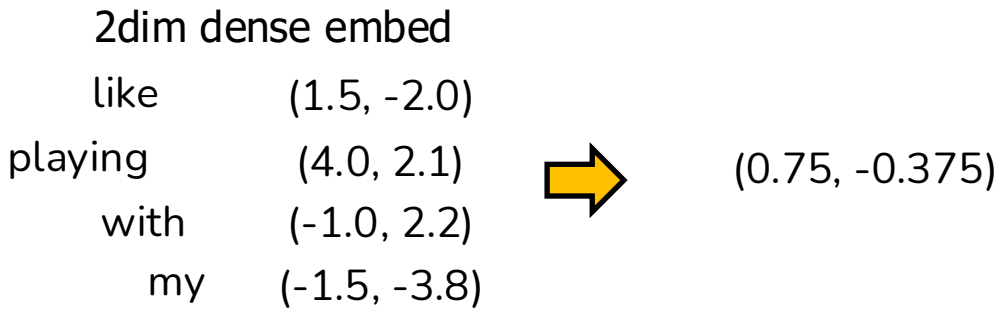
No activation in hidden layer for simplicity, speed, effectiveness

Word2Vec: CBOW (Continuous Bag of Words)

- Step 3. Forward Propagation from Input to Hidden Layer
  - Obtains word embeddings of all context words in reduced dimensions (e.g., 2) via linear feature transformation

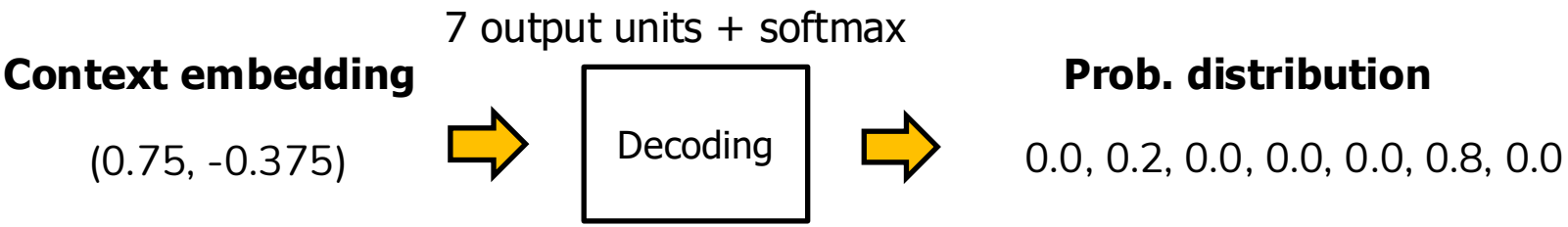


  - Merges into a single context embedding z, which is just a mean embed.



Word2Vec: CBOW (Continuous Bag of Words)

- Step 4. Forward Propagation from Hidden to Output Layer
  - Decodes the context embedding z into a vector of the same size as the input (e.g., 7 dim here)
  - Applies softmax to get a prob. distribution over all tokens



- Step 5. Backward Propagation with a "word prediction objective"

Text: I like playing football with my friend and dislike (...)

Target dist p: [football] = 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0

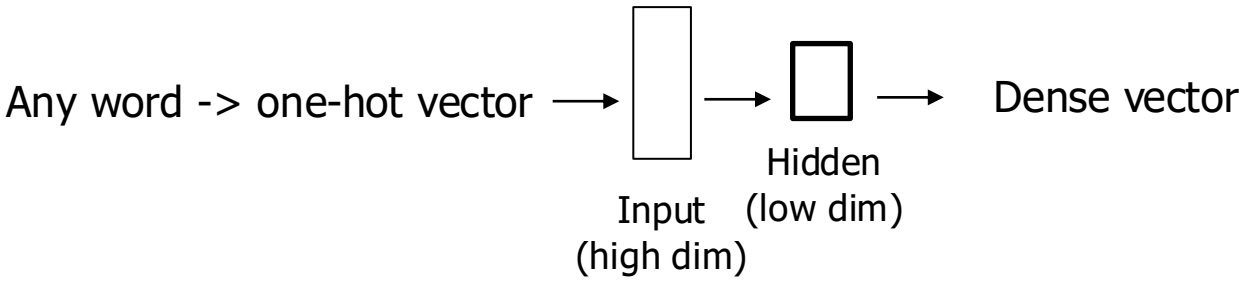
Predicted dist q = 0.0, 0.2, 0.0, 0.0, 0.0 0.8, 0.0

Cross-entropy & Backprop.

KAIST

Word2Vec: CBOW (Continuous Bag of Words)

- Step 6: Getting dense embeddings over all words
  - After training, we only use the input and hidden layers to project high dimensional one-hot vectors to lower dimensional dense vectors

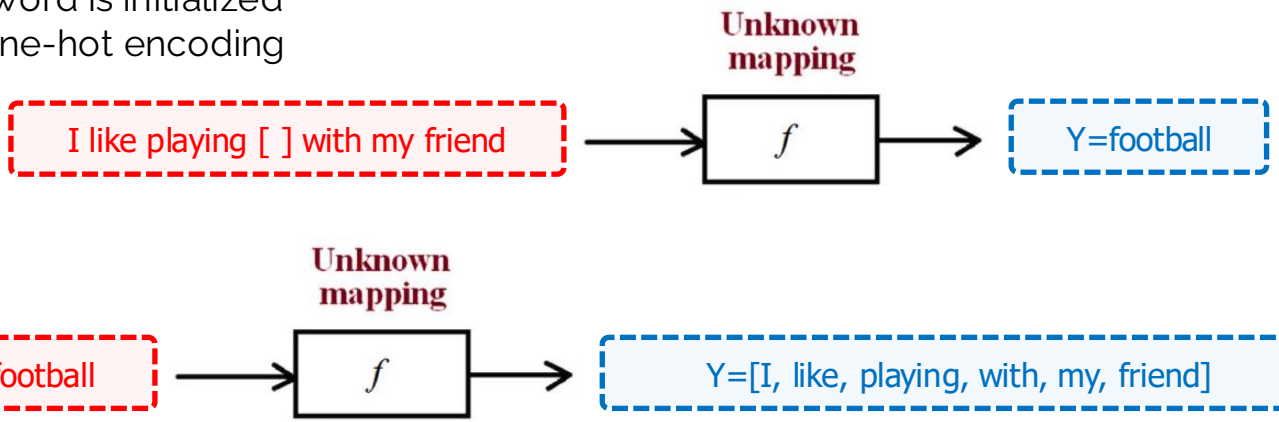


Word2Vec: Skip-gram

- Skip-gram is another way of getting dense embedding
- The main difference over CBOW is:
  - CBOW predicts the target token using other context tokens in the same window
  - Skip-gram predicts the other context tokens in the window by only using the target token

"I like playing football with my friend and dislike (…)"

(1) Each word is initialized with one-hot encoding



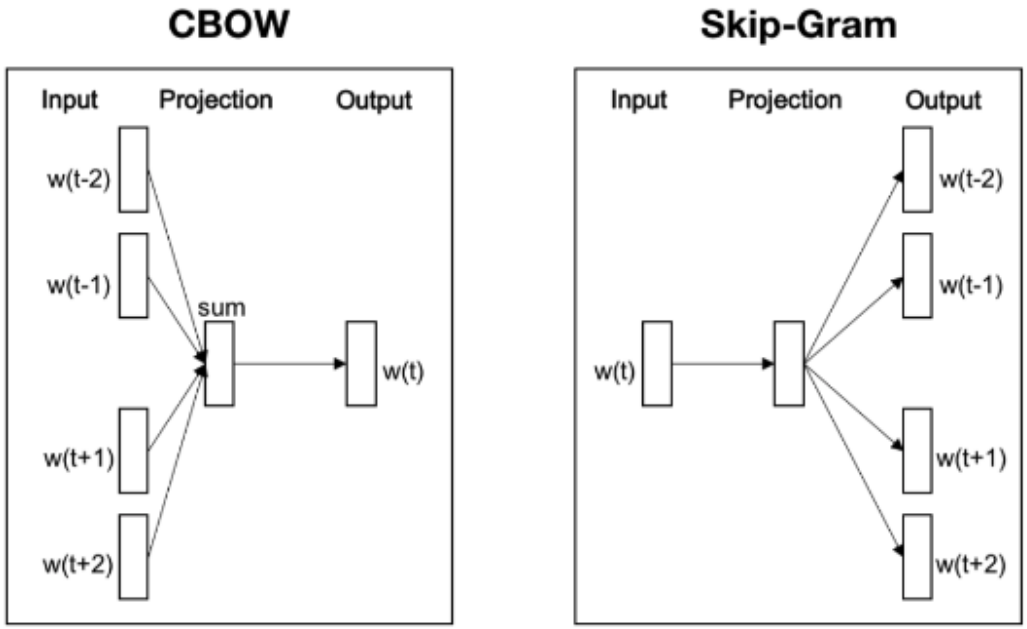
(2) A hidden layer for dimensionality reduction

(3) Token prediction with Softmax

KAIST

Word2Vec: Skip-gram

- Skip-gram is another way of getting dense embedding
- The main difference over CBOW is:
  - CBOW predicts the target token using other context tokens in the same window
  - Skip-gram predicts the other context tokens in the window by only using the target token



Word2Vec: Improved Versions

- There are some methods to get dense vectors based on W2V
- They provide the dense embeddings of most English words
- Just download and use it (see Colab)

- Word2Vec
  - https://code.google.com/archive/p/word2vec/
- Fasttext
  - https://web.stanford.edu/~jurafsky/slp3/slides/vector2.pdf
- Glove
  - https://nlp.stanford.edu/projects/glove/

Today's Contents

- Key Challenge in NLP
- Text Vectorization Basic
- Word2Vector: CBOW and Skip-gram
- **Bidirectional Autoencoding (BERT) <<**
- NLP with Pytorch

Limitation of W2V

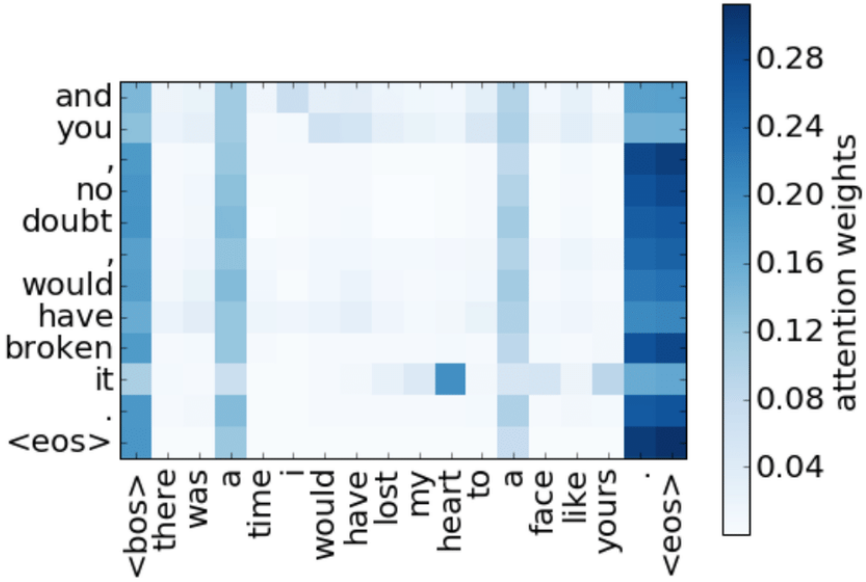
- W2V can consider an **unidirectional** relationship:
  - Either "center token to context tokens" or "context tokens to center token"
- The transformed dense embedding is **consistent** in all sentences
  - The dense embedding apple should be altered depending on context:
  - E.g., purchased an apple smartphone vs. purchased an apple at supermarket
- Window size **limits** the **context** to be considered – local semantic
  - Ignores all the tokens outside the window

With window_size of 9:

Physical activity has always been a great way for me to clear my mind and feel refreshed, especially after a long week of work or studies. Among all the activities I enjoy, one stands out more than the rest. **I like playing football with my friend and dislike** spending my time doing things that don't engage me physically. The excitement of a friendly match, the camaraderie that builds between us as we run across the field.

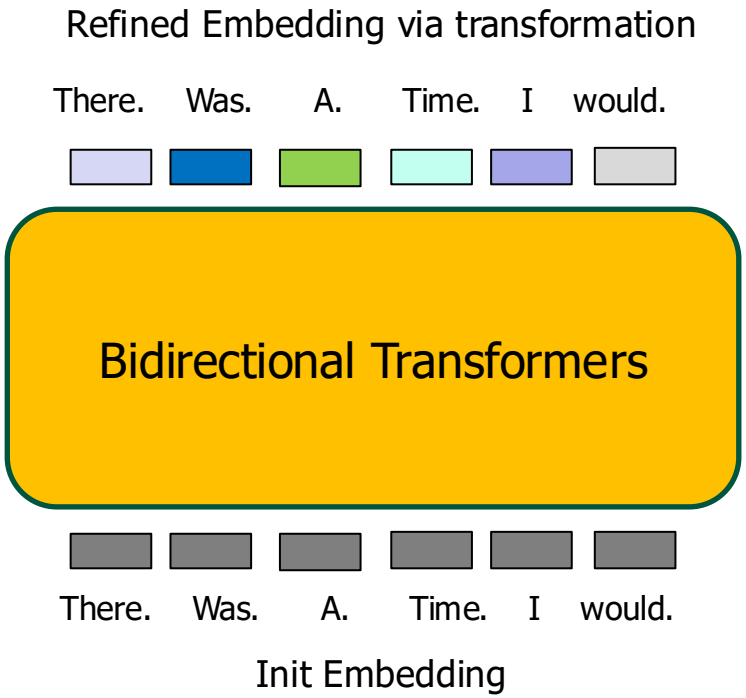
Solution: Using Attention Mechanism in Transformer

- [Recap] Attention Mechanism in Transformer
  - Capture long-term dependency and relevance of all token pairs in non-linear feature transformation using Attention Operation



Reference is the entire set of all words in the entire sequence

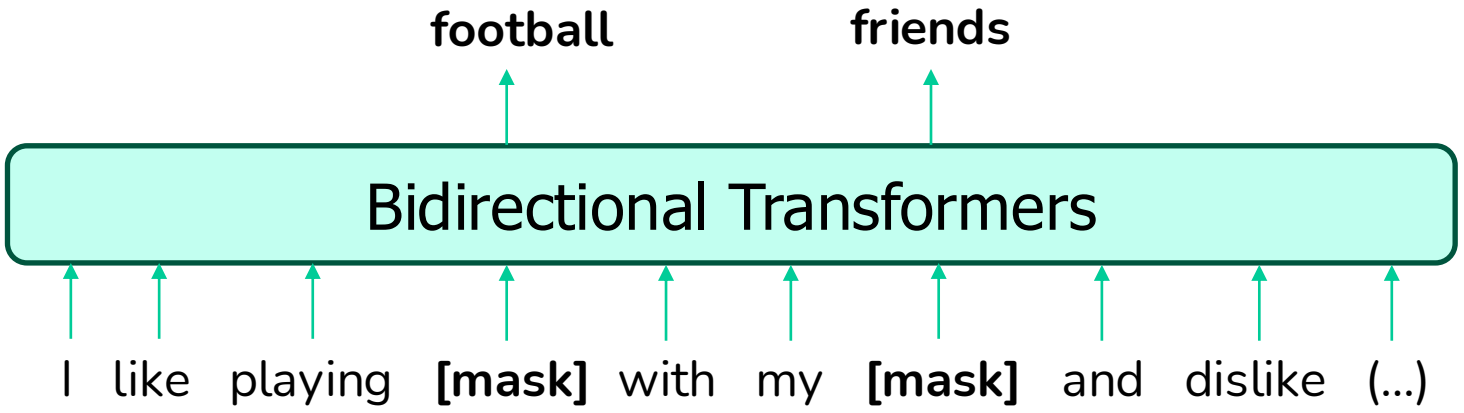
Global (Long-term) Dependency



BERT: Bidirectional Masked Autoencoding

Bert: Pre-training of deep bidirectional transformers for language understanding
J Devlin, MW Chang, K Lee, K Toutanova - arXiv preprint arXiv …, 2018 - arxiv.org
… We introduce **BERT** and its detailed implementation in this … For finetuning, the **BERT** model is first initialized with the pre-… A distinctive feature of **BERT** is its unified architecture across …
☆ Save 99 Cite Cited by 92234 Related articles All 46 versions »

- Passes all the words in the text as input to the Transformer Encoder
- Conducts a masked autoencoding, which is a task of masking a few random words from the input and then predicting the masked words
- The model learns to understand the relationships between words and the underlying context in which they appear



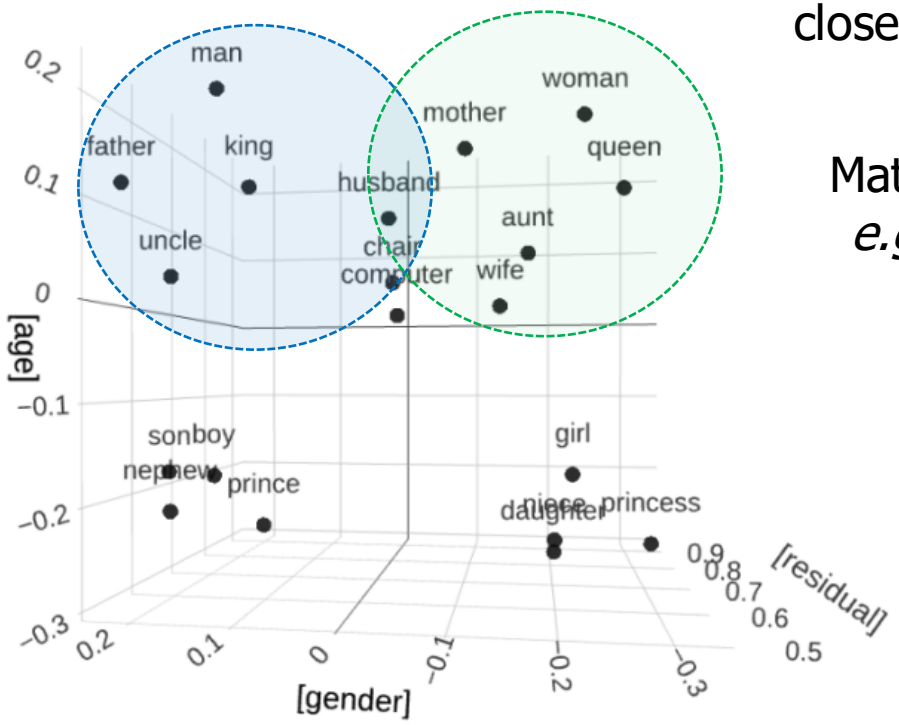
BERT: Bidirectional Masked Autoencoding

- To consider noise in natural language…
  - 15% of input text tokens are selected to swap them with …
    - 80%: Replace with [MASK] # Prediction Ability
    - 10%: Replace with "random word" # Noise Filtering
    - 10%: Keep the original word wo masking # Semantic Diff. w/wo Token

- Advantages:
  - Attention is **bidirectional** computation from left to right, right to left
  - Word embedding **varies dynamically** given context words
    - Output different embed: apple smartphone vs. apple at supermarket
  - **Global semantic** is captured with global attention

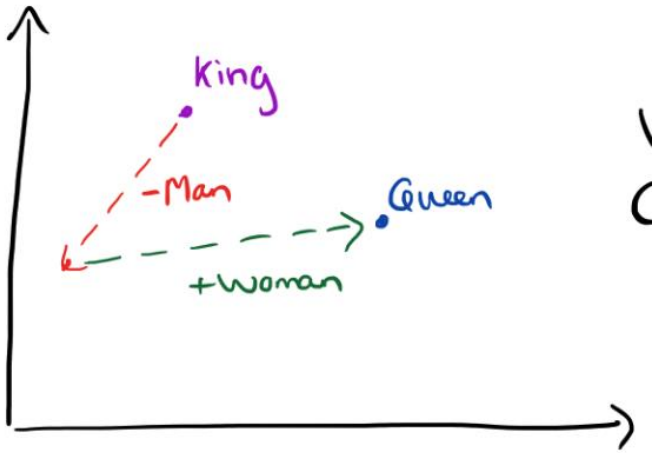
Visualization of Dense Embedding

- Dense word embedding by BERT successfully captures the semantic of different words



Words related to each gender are positioned closely together in the embedding space!

Mathematical operations are possible:
e.g., king – man + woman = queen



Today's Contents

- Key Challenge in NLP
- Text Vectorization Basic
- Word2Vector: CBOW and Skip-gram
- Bidirectional Autoencoding (BERT)
- **NLP with Pytorch <<**

- **Task 1: Processing Text Data with W2V**
  - Dense vector is a low dimensional vectors than bag-of-word and one-hot embedding, which is much easier to use as feature in machine learning
  - Glove is one of the W2V methods showing better performance than original BOW and Skip-gram: https://nlp.stanford.edu/projects/glove/
- Download Glove Pre-trained Word Embeddings
  - 6B, 42B, 840B depending on training corpora (wiki, crawl, twitter, etc)
  - For 6B version, 50dim, 100d, 200d, 300d available

```
[ ]  # recent torchtext has a bug, so use old one.
     !pip uninstall -y torchtext
     !pip install torchtext==0.5.0
```

```
[ ]  # we use 'pytorch' library to map the word into the dense vector
     import torch
     import torchtext

     # download glove embedding: the version of Glove trained on Wikipedia 2014 corpus (6B data)
     # set the size of dense embeddings to be 50
     # this will take around 5 minutes to download
     glove = torchtext.vocab.GloVe(name="6B", dim=50)
```

```
print("# of unique words:", len(glove))
```

```
# of unique words: 400000
```

NLP with Pytorch

- Let's get a word embedding of "cat"

```
[7]  # Let's look at what the embedding of the word "cat" looks like:
     print('# of words in Glove:', len(glove))
     cat = glove["cat"]
     print(cat)
```

```
# of words in Glove: 400000
tensor([ 0.4528, -0.5011, -0.5371, -0.0157,  0.2219,  0.5460, -0.6730, -0.6891,
         0.6349, -0.1973,  0.3368,  0.7735,  0.9009,  0.3849,  0.3837,  0.2657,
        -0.0806,  0.6109, -1.2894, -0.2231, -0.6158,  0.2170,  0.3561,  0.4450,
         0.6089, -1.1633, -1.1579,  0.3612,  0.1047, -0.7832,  1.4352,  0.1863,
        -0.2611,  0.8328, -0.2312,  0.3248,  0.1449, -0.4455,  0.3350, -0.9595,
        -0.0975,  0.4814, -0.4335,  0.6945,  0.9104, -0.2817,  0.4164, -1.2609,
         0.7128,  0.2378])
```

- Measuring the distance between two words (w. euclidean distance)

Measuring the distance between two words

To explore the structure of the embedding space, it is necessary to introduce a notion of distance. The Euclidean distance of two vectors x=[x1, x2, ...] and y=[y1, y2, ...] is just the 2-norm of their difference x - y. We can compute the Euclidean distance between x and y: root(sum((xi-yi)^2))

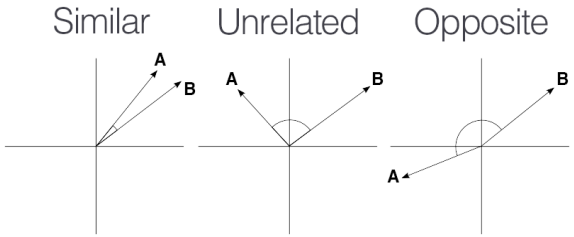
```
[ ]  # Measuring the distance between two words, cat and dog
     x = glove["cat"]
     y = glove["dog"]

     distance = torch.norm(y-x)

     # The larger the distance, the "farther" the word embeddings are to each other
     print("dog-cat:", distance.item())
```

```
dog-cat: 1.8846030235290527
```


KAIST



$$||x||_2 = \sqrt{\sum_{i=1}^{n} |x_i|^2}$$

- With Cosine distance:
  - Cosine similarity has the values in [-1.0, 1.0]; 1.0 means exactly the same; -1.0 exactly opposite

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

  - Cosine distance = 1.0 – cosine similarity; but mathematically not distance since it does not strictly follows triangle inequality;
  - Cosine distance is often more suitable than Euclidean distance in high-dimensional spaces because it ignores the magnitude of the vectors and focuses solely on the angle between them.

```
[10] # Measuring the distance using Cosine Similarity
     x = glove['cat']
     y = glove['dog']
     distance = 1.0 - torch.cosine_similarity(x.unsqueeze(0), y.unsqueeze(0))

     # The larger the similarity, the "closer" the word embeddings are to each other
     print("dog-cat:", distance.item())

     # the value in the range of
```

```
dog-cat: 0.07819944620132446
```

Dog, Cat are very similar w.r.t cosine distance

NLP with Pytorch

- Similarities between multiple word pairs:
  - dog, bike, kitten, puppy, kite, computer, neuron

```
# Let's compute the similarity between words
word = 'cat'
other = ['dog', 'bike', 'kitten', 'puppy', 'kite', 'computer', 'neuron']
for w in other:
    dist = torch.norm(glove[word] - glove[w]) # euclidean distance
    print("distance from cat to", w, float(dist))
```

```
distance from cat to dog 1.8846030235290527
distance from cat to bike 5.048375129699707
distance from cat to kitten 3.5068607330322266
distance from cat to puppy 3.0644657611846924
distance from cat to kite 4.210376262664795
distance from cat to computer 6.0306525230407715
distance from cat to neuron 6.228668212890625
```

- Finds the closest word from "cat"

```
# for example, we can look for words that are closest to another word like "cat", i.e., dog

def print_closest_words(vec, n=5):
    dists = torch.norm(glove.vectors - vec, dim=1)     # compute distances to all words
    lst = sorted(enumerate(dists.numpy()), key=lambda x: x[1]) # sort by distance
    for idx, difference in lst[1:n+1]:                 # take the top n
        print(glove.itos[idx], difference)

# retrievaing top 10 words closest to "cat" in embedding space
print_closest_words(glove["cat"], n=10)
```

```
dog 1.884603
rabbit 2.45728
monkey 2.8102055
cats 2.8972247
rat 2.945535
beast 2.9878407
monster 3.0022194
pet 3.0396755
snake 3.0617995
puppy 3.0644658
```

NLP with Pytorch

- Applies mathematical operations and finds the closest of it

Other surprising aspect of word embedding is that the mathmatical operation in the embedding space can be meaningful.

```
print_closest_words(glove['king'] - glove['man'] + glove['woman'])
```

```
queen 2.8391209
prince 3.661004
elizabeth 3.715252
daughter 3.8317878
widow 3.8493776
```

```
[ ] print_closest_words(glove['queen'] - glove['woman'] + glove['man'])
```

```
king 2.8391209
prince 3.2508988
crown 3.4485192
knight 3.5587437
coronation 3.6198907
```


KAIST

NLP with Pytorch

- Task 2: Processing Text Data with BERT
  - BERT is a deep learning model to learn better word embedding via bi-directional masked auto-encoding using Transformer Encoder
  - W2V: Downloads all word embeddings -> no semantic consideration
  - BERT: Loads the model and inference -> semantic difference reflected
- Pre-trained BERT-base Model from HuggingFace (base, large, etc)

```
[ ] # importing libraries
    import random
    import torch
    from transformers import BertTokenizer, BertModel
    from sklearn.metrics.pairwise import cosine_similarity
```

```
total_params = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total_params}")
```

```
Total parameters: 109482240
```

0.1B model

```
[ ] # Load BERT tokenizer and model
    tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
    model = BertModel.from_pretrained('bert-base-uncased')
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.51kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 4.92MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 6.07MB/s]
config.json: 100% 570/570 [00:00<00:00, 23.3kB/s]
/usr/local/lib/python3.10/dist-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean_up_tokenization_spaces` was r
  warnings.warn(
model.safetensors: 100% 440M/440M [00:04<00:00, 94.2MB/s]
```

NLP with Pytorch

- Getting embeddings of "I like dog."

```
# Input text
text = "I like dog."
```

  - Token "encoding" process from text to numeric vector (but not final embed.)
    - Step 1. Tokenization from the input text to tokens
    - Step 2. Mapping to initial text embeddings from the tokenizer

```
# Tokenize and encode text using batch_encode_plus
# The function returns a dictionary containing the token IDs and attention masks
encoding = tokenizer.batch_encode_plus(
    [text],                    # List of input texts
    padding=True,              # Pad to the maximum sequence length
    truncation=True,           # Truncate to the maximum sequence length if necessary
    return_tensors='pt',      # Return PyTorch tensors
    add_special_tokens=True    # Add special tokens CLS and SEP
)

input_ids = encoding['input_ids']  # Token IDs
attention_mask = encoding['attention_mask']  # Attention mask
print('token indexes:', input_ids)
tokens = tokenizer.convert_ids_to_tokens(tokenizer.encode(text))
print('tokenization:', tokens)
```

```
token indexes: tensor([[ 101, 1045, 2066, 3899, 1012,  102]])
tokenization: ['[CLS]', 'i', 'like', 'dog', '.', '[SEP]']
```

  - Inference with BERT

```
# Generate embeddings using BERT model
with torch.no_grad():
    outputs = model(input_ids, attention_mask=attention_mask)
    word_embeddings = outputs.last_hidden_state[0]  # This contains the embeddings

# Output the shape of word embeddings
print(f"Shape of Word Embeddings: {word_embeddings.shape}")
```

```
Shape of Word Embeddings: torch.Size([6, 768])
```


KAIST

NLP with Pytorch

- Word embedding vs. Sentence (or paragraph, document) embedding
  - Note that we can apply mathematical operation; why operation we need to use to combine all meaning of the words?

```
# Generating a sentence embedding by averaging all the token embeddings
print(f"Shape of Word Embeddings: {word_embeddings.shape}")

sentence_embedding = torch.mean(word_embeddings, axis=0)
print(f"Shape of Sentence Embedding: {sentence_embedding.shape}")
```

```
Shape of Word Embeddings: torch.Size([6, 768])
Shape of Sentence Embedding: torch.Size([768])
```

NLP with Pytorch

- Let's find the semantically the most similar sentence given a target sentence

```
# Let's compute the similarity between two sentences
text1 = "BERT embeddings are very useful for NLP tasks."
text2 = "Transformer-based embeddings like BERT help in natural language understanding."
text3 = "I like playing online games."

def get_sentence_embed(text):

  tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
  encoding = tokenizer.batch_encode_plus(
      [text],                      # List of input texts
      padding=True,                # Pad to the maximum sequence length
      truncation=True,             # Truncate to the maximum sequence length if necessary
      return_tensors='pt',        # Return PyTorch tensors
      add_special_tokens=True      # Add special tokens CLS and SEP
  )

  input_ids = encoding['input_ids']  # Token IDs
  attention_mask = encoding['attention_mask']  # Attention mask

  with torch.no_grad():
      outputs = model(input_ids, attention_mask=attention_mask)
      word_embeddings = outputs.last_hidden_state[0]
      sentence_embedding = torch.mean(word_embeddings, axis=0)

  return sentence_embedding

embed_text1 = get_sentence_embed(text1)
embed_text2 = get_sentence_embed(text2)
embed_text3 = get_sentence_embed(text3)
```

w. Euclidean distance

```
dist_btw_12 = torch.norm(embed_text1 - embed_text2, dim=0)
dist_btw_13 = torch.norm(embed_text1 - embed_text3, dim=0)

print("distance btw text 1 and 2:", dist_btw_12)
print("distance btw text 1 and 3:", dist_btw_13)
```

```
distance btw text 1 and 2: tensor(6.0784)
distance btw text 1 and 3: tensor(10.4539)
```

- Task 3: Let's use SOTA Text Embedding models
  - **How do we know what is SOTA? [Go to HuggingFace!]**
    - NV-Embed-v2 (NVIDIA model) is the SOTA and open-source for free…
    - But it is too big (7.85B), so let's use "stella_en_**1.5B**_v5"

Rank	Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Embedding Dimensions	Max Tokens	Average (56 datasets)	Classification Average (12 datasets)
1	NV-Embed-v2	7851	29.25	4096	32768	72.31	90.37
2	bge-en-icl	7111	26.49	4096	32768	71.67	88.95
3	stella_en_1.5B_v5	1543	5.75	8192	131072	71.19	87.63
4	SFR-Embedding-2_R	7111	26.49	4096	32768	70.31	89.05
5	gte-Qwen2-7B-instruct	7613	28.36	3584	131072	70.24	86.58
6	stella_en_400M_v5	435	1.62	8192	8192	70.11	86.67
7	stella_en_400M_v5	435	1.62	8192	512	70.11	86.67
8	bge-multilingual-gemma2	9242	34.43	3584	8192	69.88	88.08
9	NV-Embed-v1	7851	29.25	4096	32768	69.32	87.35
10	voyage-large-2-instruct			1024	16000	68.23	81.49

https://huggingface.co/spaces/mteb/leaderboard

NLP with Pytorch

- Let's find the most relevant documents given a query:

```
documents = [
    "Machine learning is a field of artificial intelligence that focuses on teaching machines to learn from data.",
    "Natural Language Processing enables computers to understand human language.",
    "Deep learning uses neural networks with many layers to analyze complex data patterns.",
    "Information retrieval is the process of finding relevant information from a large database."
]
query = "How do computers understand human language?"
```

- Let's load the SOTA model (id: dunzhang/stella_en_1.5B_v5) – took 2 mins for down…

```
import torch
from transformers import AutoTokenizer, AutoModel
import torch.nn.functional as F

# Load a state-of-the-art sentence embedding model from Hugging Face
model_name = "dunzhang/stella_en_1.5B_v5"  # SOTA for embedding tasks
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
model = AutoModel.from_pretrained(model_name, trust_remote_code=True)
```

```
total_params = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total_params}")
```

```
Total parameters: 1543268864
```

- Makes a function to get the embedding of 'query' and 'document' using "mean"

```
[ ] # Function to get sentence embeddings
    def get_embedding(text):
        inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)
        with torch.no_grad():
            outputs = model(**inputs)
        # Get the mean of token embeddings for a simple sentence embedding
        embedding = outputs.last_hidden_state.mean(dim=1)
        return embedding
```


KAIST

NLP with Pytorch

- Extracts all the embeddings via inference with the model (forward prop.)

```python
# Sample documents and a query
documents = [
    "Machine learning is a field of artificial intelligence that focuses on teaching machines to learn from data.",
    "Natural Language Processing enables computers to understand human language.",
    "Deep learning uses neural networks with many layers to analyze complex data patterns.",
    "Information retrieval is the process of finding relevant information from a large database."
]
query = "How do computers understand human language?"

# Generate embeddings for documents
document_embeddings = [get_embedding(doc) for doc in documents]

# Generate embedding for query
query_embedding = get_embedding(query)
```

Noted: free-tier Colab is slow

- Finds the most relevant doc to the query

w. Cosine distance

```python
# Calculate cosine similarity between query and each document
similarities = [1.0-F.cosine_similarity(query_embedding, doc_emb, dim=1).item() for doc_emb in document_embeddings]

# Find the document with the highest similarity
best_match_idx = similarities.index(min(similarities))
best_match = documents[best_match_idx]

# Display results
print(f"Query: {query}")
print(f"Best Matched Document: {best_match}")
print(f"Cosine Distance: {min(similarities):.4f}")

print('\n Optional;')
# Optional: Display similarity scores for all documents
for i, (doc, score) in enumerate(zip(documents, similarities)):
    print(f"Document {i+1} Distance: {score:.4f}")
    print(f"Document: {doc}\n")
```

```
Query: How do computers understand human language?
Best Matched Document: Natural Language Processing enables comput
Cosine Distance: 0.2591

 Optional;
Document 1 Distance: 0.6322
Document: Machine learning is a field of artificial intelligence

Document 2 Distance: 0.2591
Document: Natural Language Processing enables computers to unders

Document 3 Distance: 0.6358
Document: Deep learning uses neural networks with many layers to

Document 4 Distance: 0.7442
Document: Information retrieval is the process of finding relevar
```

Colab for Your Practice

- Colab Link:
  - https://drive.google.com/file/d/1KwBqv4NRcDH51fr7dk1BsEwxXaurRQ0Y/view?usp=sharing
- Contents:
  - Key Challenge in NLP
  - Text Vectorization
  - W2V (CBoW, Skip-gram)
  - BERT: Bi-directional Masked Auto-encoder
- Enjoy your practice on what we learn today!
  - If you have any question, email me or our teaching assistants.

Q & A